

Project Overview

This report documents the work carried out by SCAGroup6 for the SheDeploys Release Approval Board, a DevOps capstone project completed as part of the SheCodeAfrica DevOps programme. The team forked the dockersamples/example-voting-app and took it through a complete DevOps lifecycle (rebranding, containerisation, CI/CD, Kubernetes deployment, monitoring, health check and documentation).

The original Cats vs Dogs voting app was transformed into a release approval tool where engineering teams vote to Approve, Reject, or request more testing before a production deployment which is a real-world DevOps workflow pattern.

Phase 1: Fork, Rebrand & UI Changes

The team forked the dockersamples/example-voting-app repository into the SCAGroup6 GitHub organisation and rebranded it as SheDeploys, a release approval board. The original Cats vs Dogs voting options were replaced with Approve, Needs More Testing, and Reject. A timestamp backend change was implemented so every vote is recorded with the exact time it was cast, supporting audit trail requirements.

Issues Encountered

Issue 1: Wrong Port Numbers	
What happened	Assumed the app ran on ports 5000 and 5001. It actually runs on 8080 and 8081.
Root cause	Default Flask and Node.js ports were assumed without checking the Compose file.
Fix	Read docker-compose.yml — the correct ports were defined there.
Lesson	Always read the docker-compose.yml before assuming port numbers.

Issue 2: VS Code Linting Error on AngularJS Expressions	
What happened	Writing style="width: {{value}}%" showed a linting error in VS Code.
Root cause	Inline style binding is not the correct AngularJS pattern.
Fix	Used the ng-style directive instead, which is the correct AngularJS approach.

Lesson	Use framework-native directives rather than inline template expressions for style binding.
---------------	--

Issue 3: Postgres Table Not Updating With New Columns

What happened	After updating the worker to save voter_name and voted_at, the table still only had 2 columns.
Root cause	The Docker volume was preserving the old table schema from a previous run.
Fix	Added ALTER TABLE ADD COLUMN IF NOT EXISTS to the worker. Also manually ran the ALTER commands directly on the running database container.
Lesson	When changing database schemas, always run docker compose down -v to wipe the volume before restarting.

Issue 4: Voter Names Not Showing in Results Feed

What happened	The recent votes feed showed empty names even after the new columns were added.
Root cause	The worker had not been updated to read voter_name from the Redis payload — it was only writing to the new column, not populating it.
Fix	Updated worker/Program.cs to correctly read voter_name from the JSON payload and save it alongside voted_at to Postgres.
Lesson	A schema change and a data flow change must be made together. Adding a column does not automatically populate it.

Issue 5: Browser Asking to Resubmit Form on Refresh

What happened	Refreshing the vote page after voting triggered a browser prompt asking to resubmit form data.
Root cause	The vote route returned an HTML response directly after a POST request, which browsers cache as resubmittable.

Fix	Implemented the POST-Redirect-GET pattern in app.py — after saving a vote, Flask now redirects to a GET request instead of returning HTML directly.
Lesson	Always use POST-Redirect-GET for form submissions to prevent browser resubmission prompts.

Issue 6: Flask Route Not Registering

What happened	The /clear-cookie route was accidentally indented inside the if <code>__name__ == '__main__':</code> block and was never registered by Flask.
Root cause	Indentation error caused the route decorator to be inside a conditional block that only runs when the script is executed directly.
Fix	Moved the route definition outside and above the if <code>__name__ == '__main__':</code> block.
Lesson	Flask routes must be defined at module level, not inside conditional blocks. Always check indentation carefully.

Issue 7: PR Pointing to Wrong Repository

What happened	GitHub automatically set the base repository to dockersamples/example-voting-app instead of the group repo.
Root cause	When opening a PR from a fork, GitHub defaults to the upstream original repo as the base.
Fix	Navigated directly to github.com/SCAGroup6/example-voting-app/compare/dev...feature/ui-rebrand to force the correct base repository and branch.
Lesson	Always verify the base repository and branch when opening a PR from a forked repo.

Issue 8: 403 Permission Denied on Git Push

What happened	Push was rejected with a 403 permission denied error.
Root cause	The GitHub account had not been added as a collaborator to the group repository.

Fix	Group leader added the account as a collaborator. Git identity was also set locally and old stored credentials were cleared.
Lesson	Confirm collaborator access is set up before starting any work on a shared repository.

Phase 2: — CI/CD Pipeline (GitHub Actions)

This phase involved setting up a GitHub Actions CI/CD pipeline to automatically build and push Docker images for the vote, result, and worker services whenever changes are merged to the dev branch. The forked repository already contained three workflow files from the original dockersamples project, which required significant modification rather than creating new workflows from scratch.

Issues Encountered

Issue 1: Changes Not Applied Consistently Across All Three Workflow Files	
What happened	The forked repo contained three separate workflow files: call-docker-build-vote.yaml, call-docker-build-result.yaml, and call-docker-build-worker.yaml. Changes were made to one or two files but not all three, causing inconsistent pipeline behaviour.
Root cause	Each service has its own independent workflow file. It was easy to update one or two and forget the third, especially since the files look nearly identical.
Fix	Reviewed all three files side by side after every change to ensure consistency. All modifications — branch trigger, image name, authentication, permissions — were replicated across all three files.
Lesson	When working with multiple near-identical workflow files, always do a final check across all of them before committing.

Issue 2: Reusable Workflow Preventing Pipeline from Running	
What happened	The original workflow files used a reusable workflow from the dockersamples organisation: uses: dockersamples/.github/.github/workflows/reusable-docker-build.yaml@

	main. The pipeline was not running as expected and secrets were not being passed correctly.
Root cause	GitHub restricts how secrets are inherited when calling reusable workflows owned by external organisations. The reusable workflow also introduced additional complexity around credentials, image names, and build parameters that made it harder to debug and customise.
Fix	Replaced the reusable workflow entirely with a direct workflow using standard GitHub Actions steps — docker/login-action for authentication and docker/build-push-action for building and pushing images. This gave the team full control over every step.
Lesson	Reusable workflows from external organisations reduce visibility and control. For a team learning DevOps, direct workflows are easier to understand, debug, and customise.

Issue 3: Docker Hub Authentication Failing with Personal Account

What happened	DOCKERHUB_USERNAME and DOCKERHUB_TOKEN were added to the repository secrets using a personal Docker Hub account. The pipeline consistently failed at the Login to DockerHub step with: Error: Username and password required
Root cause	Two factors: (1) GitHub deliberately blocks repository secrets from being passed to workflows triggered by pull_request events from forks — a security feature to prevent malicious forks accessing credentials. (2) The personal Docker Hub account may not have had the correct permissions for the organisation's use case.
Fix	The team lead created a Docker Hub account at the organisation level and added those credentials as repository secrets. The pull_request trigger was removed from all workflows — pipelines now only trigger on push to dev (after a PR is merged), at which point secrets are available.
Lesson	For team projects, use shared organisational credentials rather than personal accounts. Always use push triggers (not pull_request) for workflows that require secrets.

Issue 4: Missing permissions Block for Docker Buildx

What happened	The pipeline was failing silently or not completing the build step correctly. Investigation revealed the workflow was missing a permissions block required for Docker Buildx to function correctly.
Root cause	Docker Buildx requires specific permissions to be declared in the workflow. The packages: write permission is specifically needed to push images to GitHub Container Registry (GHCR). Without it, authentication succeeds but the push is denied — making it look like an authentication failure.
Fix	Added the permissions block to all three workflow files: contents: read, packages: write, pull-requests: write. The pipeline ran successfully after this change was merged.
Lesson	Always include an explicit permissions block in any workflow that pushes to GHCR. The error it produces closely resembles an auth failure, which makes it easy to misdiagnose.

Issue 5: Existing CI Workflows Already Present in Forked Repo

What happened	The team attempted to create a new ci.yml workflow file, not realising the repo already contained three workflow files carried over from the original dockersamples project.
Root cause	The original example-voting-app ships with pre-built reusable workflow files. These were not noticed during the initial repo exploration.
Fix	Abandoned the new ci.yml approach and modified the existing workflow files instead — updating branch triggers from main to dev and updating image names from dockersamples to the team's own registry.
Lesson	Always inspect what already exists in a forked repo before creating new files. Read before you write.

Issue 6: Wrong Image Tag in Vote Workflow

What happened	call-docker-build-vote.yaml was pushing to paranormalrave/shedeploys-result:latest instead of paranormalrave/shedeploys-vote:latest.
----------------------	--

Root cause	Copy-paste error when updating image names across three files. The result image tag was pasted into the vote workflow without being changed.
Fix	Updated call-docker-build-vote.yaml to use the correct image tag paranormalrave/shedeploys-vote:latest.
Lesson	When making the same type of change across multiple files, review each file individually after editing.

Phase 3: Docker Compose Local Deployment

This phase involved ensuring the customised application runs correctly in containers using Docker Compose. The repo already contained Dockerfiles and a docker-compose.yml for all five services. The primary task was to verify the full stack builds and runs after the UI and backend changes from Phase 1, debug any container issues, and prepare the setup for the CI/CD pipeline.

One configuration change made in this phase was adding restart: always to the docker-compose.yml file for all services. This policy instructs Docker to automatically restart a container regardless of the exit code whether due to a crash, normal completion, or Docker daemon restart improving the resilience of the local stack.

Steps Taken

- Switched to the correct branch on the organisation repo
- Reviewed the PR files changed tab to understand what had been modified
- Cloned the repo locally and checked out the feature branch
- Reviewed the existing Dockerfiles and docker-compose.yml to understand the existing structure
- Ran docker-compose up --build and verified all 5 containers started
- Opened localhost:8080 (vote) and localhost:8081 (result) to confirm the rebranded UI loaded
- Verified all 5 containers were running using docker ps (vote, result, worker, redis, db)
- Used docker-compose logs and docker-compose logs vote to debug container issues

Issues Encountered

Issue 1: Confusion With GitHub's Automatic Patch Branches

What happened	When editing files directly on GitHub, the platform creates a new patch branch for every commit. This caused confusion about which branch contained the correct changes.
Root cause	GitHub's web editor automatically creates patch branches when you edit files without switching to a named branch first.
Fix	Switched to VS Code to make all file modifications locally, then pushed from the correct branch.
Lesson	Always make changes locally on a named feature branch rather than using GitHub's web editor to avoid unintended patch branches.

Issue 2: Accidentally Pushing to Main Instead of Fork

What happened	Could not create a pull request because changes were being pushed to the main branch of the upstream repo rather than the forked repo.
Root cause	The remote origin was pointing to the wrong repository.
Fix	Corrected the remote origin to point to the forked repo, successfully pushed to the feature branch, and opened a PR for the team lead to merge.
Lesson	Always verify your git remote -v output before pushing. Confirm origin points to your fork, not the upstream.

Phase 4: Kubernetes Deployment (Minikube)

This phase involved deploying the containerised voting app to a local Kubernetes cluster using Minikube. The repository already contained Kubernetes manifests in the k8s-specifications/ folder. The primary task was updating these manifests to reference the team's custom Docker images and resolving issues with pods serving outdated container images after code changes.

Issues Encountered

Issue 1: Pods Running Stale Container Images

What happened	Despite rebuilding Docker images and redeploying, pods continued displaying the old user interface.
----------------------	---

Root cause	Kubernetes does not automatically detect changes in locally rebuilt images unless explicitly triggered. The deployment configuration was unchanged, so no new rollout was initiated.
Fix	Forced a deployment rollout using <code>kubectl rollout restart deployment vote</code> . Reapplied all manifests with <code>kubectl apply -f .</code> and deleted stale pods with <code>kubectl delete pods --all</code> .
Lesson	Kubernetes is declarative — it only redeploys when the desired state changes. Always trigger a rollout explicitly after rebuilding images.

Issue 2: Image Pull Policy Misconfiguration

What happened	Kubernetes was reusing cached images instead of pulling updated ones, even after new images were built.
Root cause	Missing or incorrect <code>imagePullPolicy</code> setting in the deployment manifest. Without this, Kubernetes defaults to <code>IfNotPresent</code> , meaning it will use a cached image if one exists.
Fix	Set <code>imagePullPolicy: Never</code> in the deployment manifests for local Minikube development (since images are built locally, not pulled from a registry).
Lesson	Always set <code>imagePullPolicy</code> explicitly. Use <code>Never</code> for local Minikube development, <code>Always</code> for production or registry-based deployments.

Issue 3: Service Routing Traffic to Outdated Pods

What happened	The service endpoint was accessible but routing traffic to pods running outdated containers.
Root cause	Backend pods were not refreshed after updates. The service correctly selects pods by label, but those pods were still running old images.
Fix	Verified service routing with <code>kubectl get services</code> and <code>minikube service vote --url</code> . Removed stale pods with <code>kubectl delete pods --all</code> to force recreation with updated images.
Lesson	Services don't manage pod updates — they only expose existing pods. You must update the pods separately.

Phase 5: Monitoring & Health Check Script

This phase covered two deliverables: a Bash health-check script to verify all five containers are running and responsive, and the addition of Prometheus and Grafana for metrics and dashboards.

Issues Encountered — Bash Health Check Script

Issue 1: Port Conflict With Existing Prometheus and Grafana Containers	
What happened	Running docker compose up failed because Prometheus and Grafana containers from a previous session were already running and occupying the same ports.
Root cause	Previously started monitoring containers were not stopped before attempting to bring up a new stack.
Fix	Removed the conflicting containers with <code>docker rm -f prometheus</code> and <code>docker rm -f grafana</code> , ran <code>docker compose down</code> , then restarted cleanly.
Lesson	Always run <code>docker compose down</code> before starting a new session, especially when monitoring services are involved.

Issue 2: Database Credentials Not Configured	
What happened	Running <code>docker compose ps</code> showed the worker service was unhealthy due to a Postgres authentication error.
Root cause	The database username and password had not been explicitly configured. The default Postgres credentials needed to be either set or explicitly confirmed in the Compose file.
Fix	Confirmed the default Postgres credentials (<code>POSTGRES_USER: postgres</code> , <code>POSTGRES_PASSWORD: postgres</code>) were in place in the <code>docker-compose.yml</code> environment section.
Lesson	Always verify database credentials in the Compose file before running the stack. The worker logs will show auth errors immediately if there is a mismatch.

Issue 3: Logic Errors in Bash Health Check Script

What happened	The health check script was not executing correctly due to logical issues in the conditional checks.
Root cause	The if-else conditions comparing expected vs actual container counts and the for loop iterating over running containers contained logic errors that prevented correct execution.
Fix	Rewrote the conditional logic with correct if-else statements and a properly structured for loop to iterate over running containers and display health status per service.
Lesson	Test Bash scripts incrementally — run each section independently before combining into a full script.

Issue 4: Script Permission Issue	
What happened	The health check script could not be executed directly due to missing execute permissions.
Root cause	The script file was created without the execute permission bit set.
Fix	Used <code>bash healthcheck.sh</code> instead of <code>./healthcheck.sh</code> , which invokes the Bash interpreter directly and does not require execute permissions on the file.
Lesson	Use <code>bash <filename></code> during development and testing. Add <code>chmod +x</code> only when the script is finalised and needs to be run as an executable.

Phase 6: Terraform AWS Production Infrastructure

This phase involved writing Terraform to provision the full production infrastructure on AWS, replacing the local Docker Compose stack with cloud-native equivalents: VPC (networking), ECS Fargate (containers), RDS PostgreSQL (database), ElastiCache Redis (caching), Application Load Balancer (traffic routing), security groups (access control), and IAM roles (permissions).

Infrastructure Provisioned

- VPC with public and private subnets across two availability zones
- ECS Fargate cluster with task definitions for vote, result, and worker services
- RDS PostgreSQL instance in a private subnet (replacing local Postgres container)
- ElastiCache Redis cluster (replacing local Redis container)

- Application Load Balancer with target groups for vote (port 80) and result (port 8080)
- Security groups controlling traffic between ALB, ECS tasks, RDS, and ElastiCache
- IAM roles for ECS task execution
- CloudWatch alarms for infrastructure monitoring (additional deliverable)

Issues Encountered

Issue 1: Result App Not Accessible After Deployment	
What happened	After deployment, the result app (port 8080) was not accessible and kept timing out.
Root cause	The load balancer security group was only allowing inbound traffic on port 80. Port 8080 had not been added as an allowed inbound rule.
Fix	Added port 8080 to the ALB security group in Terraform and re-ran terraform apply.
Lesson	When deploying multi-port applications behind an ALB, explicitly add every application port to the security group inbound rules.

Issue 2: Worker Service Could Not Connect to RDS or ElastiCache	
What happened	The worker service kept logging Waiting for db and could not connect to either RDS or Redis.
Root cause	The application code used hardcoded service names db and redis for database connections — the same names used in Docker Compose DNS resolution. These names do not exist in AWS; ECS containers must use actual endpoint URLs or environment variables.
Fix	Updated the application code to read database and Redis connection details from environment variables. Terraform was configured to pass the RDS endpoint and ElastiCache endpoint into the ECS task definitions as environment variables.
Lesson	Never hardcode service names from Docker Compose in application code. Always use environment variables for connection strings so the same codebase works in both local and cloud environments.

Issue 3: Confusion Between ECS Fargate and EC2

What happened	After deploying the ECS cluster, the team looked for EC2 instances in the AWS console and found none, causing confusion about whether the deployment had succeeded.
Root cause	The team expected ECS to create visible EC2 instances as with a traditional deployment. ECS Fargate is serverless — it runs containers without provisioning or managing EC2 instances.
Fix	Checked the ECS console directly (not EC2) where all tasks and services were visible and running correctly.
Lesson	ECS Fargate abstracts away the underlying compute — there are no EC2 instances to manage. Always check the ECS console, not EC2, when using Fargate.

Issue 4: Resources Not Visible in AWS Console

What happened	After a successful terraform apply, resources could not be found in the AWS console.
Root cause	The team was viewing the wrong AWS region in the console. Resources were deployed to us-east-1 but the console was showing a different region.
Fix	Switched the AWS console region selector to us-east-1, after which all resources appeared correctly.
Lesson	Always confirm the AWS region in both your Terraform configuration and the AWS console. A region mismatch is one of the most common sources of confusion when working with AWS.

Key Learnings Across All Phases

S/N	Area	Key Learning
1	Git Workflow	Always verify git remote -v before pushing. Confirm you are on the correct branch and the correct repo.
2	Docker Compose	Run docker compose down -v when changing database schemas to wipe old volumes.

3	GitHub Actions	Use push triggers (not pull_request) for workflows that require secrets. Fork PRs block secret access by design.
4	Reusable Workflows	Reusable workflows from external orgs reduce visibility. Use direct steps when learning or debugging.
5	Multi-file Consistency	When the same change must be made across multiple files, check every file individually before committing.
6	Kubernetes	Kubernetes is declarative — it only redeploys when the desired state changes. Trigger rollouts explicitly after image updates.
7	GHCR Permissions	Always include packages: write in the permissions block when pushing to GitHub Container Registry.
8	Environment Variables	Never hardcode service names from Docker Compose in application code. Use environment variables for all connection strings.
9	AWS Region	Always confirm the AWS region matches between Terraform config and the AWS console.
10	ECS Fargate	Fargate is serverless — there are no EC2 instances. Check the ECS console, not EC2, for your running tasks.